Smart Demand Prediction

Dynamic Pricing Optimization Model

VIBIN SRI R

Problem Statement

The Challenge

1. Retailers struggle to price products correctly
2. Competitor prices constantly change
3. Difficult to predict if a product will have High or Low demand
4. Pricing decisions are made on intuition, not data

Our Solution

- ✓ Build an ML model to classify demand as High or Low
- ✓ Use real pricing, competitor & customer data
- ✓ Deploy as a live Streamlit dashboard
- ✓ Give instant predictions + price optimization

System Architecture

Outputs

`cleaned_data.csv`	`feature_engineered_data.csv`	`trained_model.pkl`	`app.py`
592 rows after removing outliers	49 features including 11 engineered	XGBoost model — 91.6% accuracy	Live Streamlit prediction app

Dataset Overview

676	**592**	**30**	**9**
Raw Records	After Cleaning	Original Features	Product Categories

Key Columns in the Dataset

- unit_price — Price per unit (R$)
- qty — Quantity sold (our target)
- total_price — Total revenue
- freight_price — Shipping cost
- customers — No. of customers
- comp_1, comp_2, comp_3 — Competitor prices
- product_score — Product rating (1–5)
- lag_price — Previous month's price
- s — Sales velocity metric
- product_category_name — Category (9 types)

Data Cleaning

Before: 676 rows, 30 columns → **After:** 592 rows, 38 columns, clean & ready

1 No Missing Values

data.isnull().sum() = 0 across all columns

2 Removed Outliers

IQR method on unit_price & qty — removed 84 rows

3 Parsed Dates

month_year → month, year, day_of_week

4 Log Transform

log1p(qty) and log1p(unit_price) to reduce skew

5 Standard Scaling

Z-score on unit_price & qty (mean=106.5, std=76.2)

6 One-Hot Encoding

9 categories → 8 binary columns (drop_first=True)

Exploratory Data Analysis

What the data tells us:

Price & Demand are Negatively Correlated

As unit price increases, quantity sold tends to decrease — classic price elasticity.

Freight Cost Impacts Sales

Higher freight price reduces demand, especially for low-margin products.

Strong Seasonal Pattern

November 2017 had the highest sales (1,051 units). Mid-2018 saw a drop.

Competitor Pricing Matters

Products priced below competitors showed significantly higher demand.

Garden Tools is the Largest Category

160 records out of 676 — 23.7% of the dataset.

Product Score Cluster around 4.0–4.2

Most products rated 4.0 or above, giving limited score variance.

Feature Engineering

Started with 38 cleaned columns → Created 11 new features → 49 total features used in model

Feature	Formula	Why it matters
`price_per_customer`	`total_price / (customers + 1)`	How much revenue each customer brings
`comp1_gap, comp2_gap, comp3_gap`	`unit_price − comp_1/2/3`	Are we cheaper or costlier than rivals?
`price_change`	`unit_price − lag_price`	Did the price go up or down this month?
`discount_flag`	`1 if price_change < 0 else 0`	Simple binary: is this a discounted price?
`quarter`	`(month − 1) // 3 + 1`	Captures seasonal buying patterns
`price_freight_interaction`	`unit_price × freight_price`	Combined cost signal for buyer
`comp×discount interactions`	`comp_gap × discount_flag`	Effect of discount relative to competitor

Model Building

Target Variable: log_qty > median(2.303) → Class 1 (High Demand), otherwise → Class 0 (Low Demand)

Logistic Regression

82.35%

Accuracy

Did not converge fully (max_iter=1000)

Random Forest

89.92%

Accuracy

Good performance, but less tunable

XGBoost

91.60%

Accuracy

Best accuracy after tuning — selected!

★ **SELECTED**

Tuned with RandomizedSearchCV (5-fold CV): n_estimators=100, max_depth=5, learning_rate=0.1, subsample=0.8 → CV Score: 90.9%

Model Results

91.6%	0.92	0.95	0.95
Test Accuracy	F1 Score	Precision (Low)	Recall (High)

Classification Report (XGBoost Tuned):

Class	Precision	Recall	F1-Score	Support
0 — Low Demand	0.95	0.88	0.91	60
1 — High Demand	0.89	0.95	0.92	59
Accuracy			**0.92**	**119**
Macro Average	0.92	0.92	0.92	119

Top 3 Features (by importance):

1. **s (sales velocity)**	**12.7%**
2. health_beauty category	**7.2%**
3. comp_3 price	**6.6%**
4. customers	**6.6%**
5. price_per_customer	**6.4%**

Streamlit App · dashboard/app.py

A live web app with 3 tabs — runs locally

Sidebar Inputs

- Your Unit Price (R$)
- Last Month Price (R$)
- Competitor 1, 2, 3 Prices
- Freight Price (R$)
- Product Score (1–5)
- No. of Customers
- Sales Velocity (s)
- Category
- Month / Day of Week

Prediction

- Enter inputs from sidebar
- Model predicts High or Low demand
- Confidence % shown via gauge
- Price optimiser sweeps 40 prices to find best revenue

Data Insights

- Price distribution by demand class
- Category breakdown chart
- Price vs Quantity scatter plot
- Competitor price boxplot
- Correlation heatmap

Model Metrics

- Accuracy, AUC, Precision, Recall
- Feature importance bar chart
- Confusion matrix heatmap
- ROC curve with AUC = 0.97

Challenges & Key Learnings

Challenges Faced

✗ Model always predicted LOW

The `s` (sales velocity) column was never being set — it is the top feature at 12.7% importance

✗ Wrong StandardScaler values

Used raw data mean/std instead of cleaned data: should be mean=106.5, std=76.2

✗ Feature formula mismatch

Training used total_price/(customers+1) but app used unit_price/customers

✗ Regression → Classification

Had to convert continuous qty prediction into binary High/Low using median threshold

Key Learnings

1. Training transformations must be reproduced exactly at inference time
2. Always check the source data for important columns before engineering new ones
3. SHAP values explain individual predictions and build trust with non-technical audiences
4. Starting simple (LR → RF → XGBoost) helps understand each model's contribution
5. End-to-end testing (train → predict → verify) is essential — unit tests alone are not enough

Thank You

Open for Questions

Smart Demand Prediction Dashboard · Capstone Project · 2026